

Programación concurrente y paralela

## **Práctica 4 - Listas y Skip Lists Concurrentes**

---

Segundo Cuatrimestre 2026



Repaso

- Buscamos definir estructuras de datos que permitan admitan alguna equivalencia con el comportamiento secuencial.
- En particular vimos la *Consistencia Secuencial*, la posibilidad de ordenar los eventos para representar comportamiento secuencial.
- También, más fuerte, la *lineazabilidad*, poder definir un momento en la ejecución de nuestras operaciones donde consideramos que se realizó la acción.

Podemos asignar el siguiente atributo a nuestras estructuras:

- Una operación es *Wait-Free* si sabemos que siempre finalizará en una cantidad acotada de pasos.
- Una estructura es *Wait-Free* si todas sus operaciones son *Wait-free*.
- Una estructura es *Lock-Free* cuando siempre hay al menos algún proceso ejecutando una operación que terminará en una cantidad acotada de pasos (“hay progreso global”).

Repasamos las diferentes implementaciones de conjuntos sobre listas:

Se aplica un sobre toda la lista para operar (se puede hacer directamente con métodos `synchronized`)

(obs: El entry set **no** es ordenado en Java, puede haber starvation)

Cada elemento tiene un Lock.

Se toman los locks del elemento a usar **y el anterior**, la toma de locks se hace con el método *hand over hand*.

No se toman los locks hasta llegar a los elementos buscados, se verifica que una vez tomados los locks se tenga consistencia recorriendo la lista nuevamente.

(obs: En este esquema hay threads que pueden quedar en starvation)



Se agrega un marcador de “borrado” a los nodos; esto evita tener que rechequear la lista para validarla (pero no impide que tengamos que volver a recorrerla si debemos reintentar).

- Idea: Puedo usar el concepto “Optimista” en granulado grueso, es decir, buscar los nodos que necesito, y luego tomar el lock de la lista.
- El problema es, ¿No hace esto que de todas maneras tenga que re-chequear una vez tomado el lock?

Podemos resolver esto con un nuevo atributo en la lista que sea el número de versión. Cada acción de agregado o borrado incrementa el número de versión.

Luego, miramos la versión antes de la búsqueda, encontramos, tomamos el lock, y chequeamos que el número de versión sea el mismo que teníamos.

La solución finalmente no gana tanto respecto al granulado grueso debido a que una modificación, aunque sea en elementos sin relación al que estamos operando, incrementa el número de versión.

Se eliminan los locks; se utiliza una instrucción atómica que, en una misma operación, verifica que una referencia es válida y a su vez chequea (y setea) el valor de un booleano: `CompareAndSet`.

Usando esta instrucción, se puede garantizar que se está actualizando un nodo no-borrado y que apunta a la referencia correcta.

# Skip Lists

- Las estructuras arbóreas que preservan altura (por ej. AVLs) tienen que hacer rotaciones incrementando la contención.
- La idea es “simular” un árbol con una lista multinivel
- Esto nos va a dar complejidad *esperada*  $O(\log n)$  (donde  $n$  es la cantidad de elementos de la lista)

Vamos a ver las ideas de dos implementaciones de Skip Lists:

- `LazySkipList`, similar a la implementación *Lazy* de listas
- `LockFreeSkipList`, similar a la implementación *Lock-Free* de listas

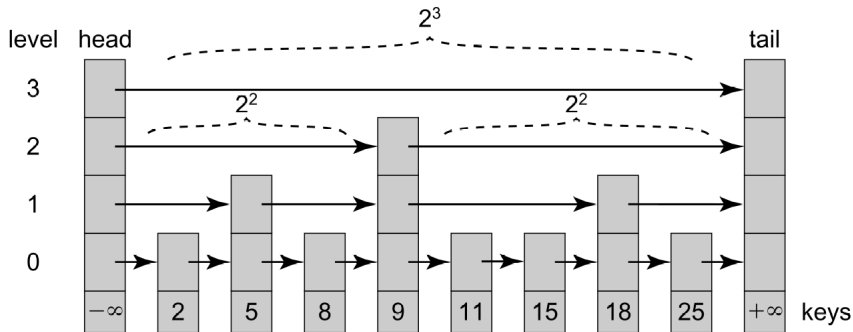
En ambas, `contains()` es wait-free

Mantenemos las reglas de la lista

- Tenemos un hash perfecto (no hay dos objetos con el mismo hash)
- La lista se mantiene ordenada por key (resultado de la f. hash)
- Tenemos nodos centinelas *head* y *tail* con key `MAX_INT` y `MIN_INT`



- Tenemos múltiples listas cada una con un *nivel*
- *skiplist property*: Toda lista del nivel  $n + 1$  está contenida en la lista de nivel  $n$
- Usamos el mismo nodo, que tiene referencias al próximo para cada nivel en el que está
- Las listas de niveles superiores funcionan como “atajos” (*skips*) para recorrer la lista completa (la del nivel 0)

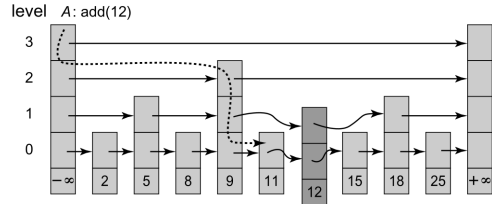
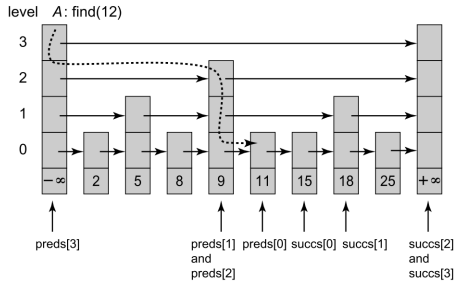


En esta versión ideal con 8 nodos, hay exactamente 1 de nivel 2 y 3 de nivel 1. Para buscar, empiezo en el nivel superior y voy bajando a medida que me voy pasando.

En la práctica, las skip lists son probabilísticas.

- Se busca que la altura  $H$  sea el logaritmo de la longitud media esperado
- Al insertar un nodo defino cuál es su nivel máximo  $i$  ( $0 < i \leq H$ )
- Fijando una probabilidad  $p$  (usualmente 0.5), el nivel máximo del nodo es  $i$  con probabilidad  $p^i$ .
- Si un nodo está en el nivel  $i$ , está en todo nivel  $0 \leq j < i$
- *head* y *tail* tienen altura  $H$ .

Para recorrer la lista nos apoyamos en un método `find`, que no sólo nos dice si el elemento pertenece a la lista, también nos dice el nivel en el que lo encontró y, más importante, sus predecesores en todos los niveles y sus sucesores en niveles más altos.



Como en la clase teórica, representamos conjuntos por medio de skip lists.

- Como en listas lazy, se toman los locks y se verifica que no hayan habido cambios
- Los nodos tienen dos booleanos: El marcador para borrado y *fullyLinked*, que indica que el nodo está seteado en todos su predecesores

Inv. Rep: Un valor está en el conjunto si y sólo si la skiplist tiene un nodo **sin marcar** y **fully linked** cuya key es el hash del valor.

```
1 public final class LazySkipList<T> {
2     static final int MAX_LEVEL = ...;
3     final Node<T> head =
4         new Node<T>(Integer.MIN_VALUE);
5     final Node<T> tail =
6         new Node<T>(Integer.MAX_VALUE);
7     public LazySkipList() {
8         for (int i = 0; i < head.next.length; i++)
9             head.next[i] = tail;
10    }
11 }
12
13 private static final class Node<T> {
14     final Lock lock = new ReentrantLock();
15     final T item;
16     final int key;
17     final Node<T>[] next;
18     volatile boolean marked = false;
19     volatile boolean fullyLinked = false;
20     private int topLevel;
```

```
21 // constructor sentinela
22     public Node(int key) {
23         this.item = null;
24         this.key = key;
25         next = new Node[MAX_LEVEL + 1];
26         topLevel = MAX_LEVEL;
27     }
28
29     public Node(T x, int height) {
30         item = x;
31         key = x.hashCode();
32         next = new Node[height + 1];
33         topLevel = height;
34     }
35
36     public void lock() {
37         lock.lock();
38     }
39
40     public void unlock() {
41         lock.unlock();
42     }
43 }
44 }
```



- Recorre la lista usando `prev` y `curr` desde el nivel más alto, sin tomar locks.
- Si el elemento no se encuentra en el nivel actual, baja (recorre desde `prev` un nivel más abajo), recordando el predecesor y el sucesor en el nivel actual (se le pasan dos arrays donde se guarda esta información).
- Si encuentra el elemento, informa el nivel donde lo encontró (i.e., la altura del elemento).
- Si llega al nivel 0 sin encontrar el elemento, devuelve `-1`.

```
1 int find(T x, Node<T>[] preds, Node<T>[] succs) {
2     int key = x.hashCode();
3     int lFound = -1;
4     Node<T> pred = head;
5     for (int level = MAX_LEVEL; level >= 0;
6         level--) {
7         volatile Node<T> curr = pred.next[level];
8         while (key > curr.key) {
9             pred = curr; curr = pred.next[level];
10        }
11        if (lFound == -1 && key == curr.key) {
12            lFound = level;
13        }
14        preds[level] = pred;
15        succs[level] = curr;
16    }
17    return lFound;
18 }
```

- Se basa en `find`: Devuelve `true` en tanto `find` encuentra el nodo, y este está `fullyLinked`, y no está marcado.
- No adquiere locks ni reinicia la búsqueda.

```
1  boolean contains(T x) {
2      Node<T>[] preds =
3          (Node<T>[]) new Node[MAX_LEVEL + 1];
4      Node<T>[] succs =
5          (Node<T>[]) new Node[MAX_LEVEL + 1];
6      int lFound = find(x, preds, succs); //linealizacion en el find
7      return (lFound != -1
8              && succs[lFound].fullyLinked
9              && !succs[lFound].marked);
10 }
```

- Usa `find` para ver si el elemento ya existe, de hacerlo y no estar marcado devuelve `false`; si cuando lo encuentra no tiene prendido el flag `fullyLinked`, **lo espera**.
- Si el elemento existe pero está marcado, alguien lo está borrando, empieza de nuevo
- Nivel por nivel (empezando del 0), toma los locks del predecesor en cada nivel, y verifica que no estén marcados y que el predecesor apunte al sucesor. Si este chequeo falla, se empieza de nuevo.
- Tomados los locks, crea el nodo, actualiza referencias, y marca el flag `fullyLinked`.
- Finalmente, libera los locks y devuelve `true`.

```
1 boolean add(T x) {
2     int topLevel = randomLevel();
3     Node<T>[] preds =
4         (Node<T>[]) new Node[MAX_LEVEL + 1];
5     Node<T>[] succs =
6         (Node<T>[]) new Node[MAX_LEVEL + 1];
7     while (true) {
8         int lFound = find(x, preds, succs);
9         if (lFound != -1) {
10             Node<T> nodeFound = succs[lFound];
11             if (!nodeFound.marked) {
12                 while (!nodeFound.fullyLinked) {}
13                 return false; //linealiza fallo
14             }
15             continue; //el nodo estaba marcado
16         }
17         int highestLocked = -1;
18         try {
19             Node<T> pred, succ;
20             boolean valid = true;
21             for (int level = 0;
22                 valid && (level <= topLevel); level++) {
23                 pred = preds[level];
```

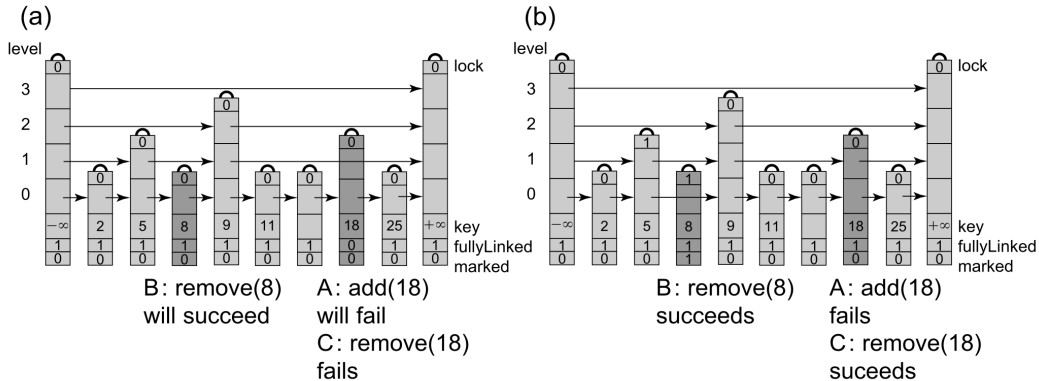
```
26         succ = succs[level];
27         pred.lock.lock();
28         highestLocked = level;
29         valid = !pred.marked
30             && !succ.marked
31             && pred.next[level]==succ;
32     }
33     if (!valid) continue;
34     Node<T> newNode = new Node(x, topLevel);
35     for (int level = 0;
36         level <= topLevel; level++)
37         newNode.next[level] = succs[level];
38     for (int level = 0;
39         level <= topLevel; level++)
40         preds[level].next[level] = newNode;
41     newNode.fullyLinked = true; //linealiza exito
42     return true;
43 } finally {
44     for (int level = 0;
45         level <= highestLocked; level++)
46         preds[level].unlock();
47     }
48 }
49 }
```

- Usa `find` para encontrar el elemento y chequea que tenga `fullyLinked`, no esté marcado, y que su `topLevel` coincida con el nivel donde `find` lo encontró. Si no, falla.
- Toma el lock del elemento, y valida que aún no esté marcado. Lo marca.
- Va subiendo nivel por nivel (hasta el nivel del elemento) tomando el lock del predecesor y verificando que este no esté marcado y que apunte al elemento a borrar. Si algo de esto falla, libera los locks de los predecesores tomados (no el del elemento a borrar) y comienza de nuevo.
- Si consigue los locks, va de arriba hacia abajo reemplazando las referencias al próximo, haciendo efectivo el borrado.
- Finalmente, libera los locks y devuelve `true`.

```
1 boolean remove(T x) {
2     Node<T> victim = null;
3     boolean isMarked = false;
4     int topLevel = -1;
5     Node<T>[] preds =
6         (Node<T>[]) new Node[MAX_LEVEL + 1];
7     Node<T>[] succs =
8         (Node<T>[]) new Node[MAX_LEVEL + 1];
9     while (true) {
10        int lFound = find(x, preds, succs);
11        if (lFound != -1) victim = succs[lFound];
12        if (isMarked ||
13            (lFound != -1 &&
14             (victim.fullyLinked
15              && victim.topLevel == lFound
16              && !victim.marked))) {
17            if (!isMarked) {
18                topLevel = victim.topLevel;
19                victim.lock.lock();
20                if (victim.marked) {
21                    victim.lock.unlock();
22                    return false; //linealiza fallo
23                }
24                victim.marked = true; //linealiza exito
25                isMarked = true;
26            }
27        }
```

```
26 int highestLocked = -1;
27 try {
28     Node<T> pred, succ; boolean valid = true;
29     for (int level = 0; valid && (level <= topLevel);
30         level++) {
31         pred = preds[level];
32         pred.lock.lock();
33         highestLocked = level;
34         valid = !pred.marked
35             && pred.next[level] == victim;
36     }
37     if (!valid) continue;
38     for (int level = topLevel; level >= 0; level--) {
39         preds[level].next[level] = victim.next[level];
40     }
41     victim.lock.unlock();
42     return true;
43 } finally {
44     for (int i = 0; i <= highestLocked; i++) {
45         preds[i].unlock();
46     }
47 }
48 } else return false; //linealiza fallo
49 }
50 }
```





- Trae a Skip List las ideas de la implementación Lock-Free de listas.
- Usa referencias atómicamente marcables y CompareAndSet.
- Como no vamos a lockear, no podemos cambiar todos los niveles a la vez y por lo tanto **romperemos la propiedad skiplist** (podemos tener nodos que saltean niveles). No hay fullyLinkedFlag.
- Inv. Rep: Un elemento está en el conjunto si y sólo si está (es alcanzable) **en el nivel 0** de la estructura y no está marcado.

- Los métodos `add` y `remove` se comportan como si cada nivel fuera su propia lista Lazy.
- El método `find` se encarga de hacer el borrado efectivo de nodos marcados

```
1 public final class LockFreeSkipList<T> {
2     static final int MAX_LEVEL = ...; //depende
3     final Node<T> head =
4         new Node<T>(Integer.MIN_VALUE);
5     final Node<T> tail =
6         new Node<T>(Integer.MAX_VALUE);
7     public LockFreeSkipList() {
8         for (int i = 0; i < head.next.length;
9             i++) {
10             head.next[i] =
11                 new AtomicMarkableReference<
12                     LockFreeSkipList.Node<T>>(tail, false);
13         }
14     }
15
16     public static final class Node<T> {
17         final T value; final int key;
18         final AtomicMarkableReference<Node<T>>[] next;
19         private int topLevel;
```

```
20 //constructor para sentinelas
21     public Node(int key) {
22         value = null; key = key;
23         next = (AtomicMarkableReference<Node<T>>[])
24             new AtomicMarkableReference[MAX_LEVEL + 1];
25         for (int i = 0; i < next.length; i++) {
26             next[i] = new AtomicMarkableReference<Node<T>>(
27                 null, false);
28         }
29         topLevel = MAX_LEVEL;
30     }
31 //constructor para nodos regulares
32     public Node(T x, int height) {
33         value = x;
34         key = x.hashCode();
35         next = (AtomicMarkableReference<Node<T>>[])
36             new AtomicMarkableReference[height + 1];
37         for (int i = 0; i < next.length; i++) {
38             next[i] = new AtomicMarkableReference<Node<T>>(
39                 null, false);
40         }
41         topLevel = height;
42     }
43 }
```

- Busca el elemento y los vectores de predecesores y sucesores con `find` (similar a la Lazy). De encontrarlo, devuelve `false`.
- Si no, crea el nodo, setea sus sucesores, e intenta agregarlo al nivel 0 con `CompareAndSet`. Si falla comienza de nuevo (encontró un nodo marcado, `find` lo eliminará).
- Hecho esto, repite con los niveles superiores (hasta el nivel asignado al nodo)

```
1 boolean add(T x) {
2     int topLevel = randomLevel();
3     int bottomLevel = 0;
4     Node<T>[] preds =
5         (Node<T>[]) new Node[MAX_LEVEL + 1];
6     Node<T>[] succs =
7         (Node<T>[]) new Node[MAX_LEVEL + 1];
8     while (true) {
9         boolean found = find(x, preds, succs);
10        if (found) { //linealización fallo
11            return false;
12        } else {
13            Node<T> newNode = new Node(x, topLevel);
14            for (int level = bottomLevel; level <=
15                topLevel;
16                 level++) {
17                Node<T> succ = succs[level];
18                newNode.next[level].set(succ, false);
19            }
20        }
21    }
22 }
```

```
19 Node<T> pred = preds[bottomLevel];
20 Node<T> succ = succs[bottomLevel];
21 if (!pred.next[bottomLevel].compareAndSet(
22     succ, newNode, false, false)) { //l. éxito
23     continue;
24 }
25 for (int level = bottomLevel+1;
26      level <= topLevel; level++) {
27     while (true) {
28         pred = preds[level];
29         succ = succs[level];
30         if (pred.next[level].compareAndSet(
31             succ, newNode, false, false))
32             break;
33         find(x, preds, succs);
34     }
35 }
36 return true;
37 }
38 }
39 }
```

- Primero, invoca a `find` para determinar la existencia del elemento en el nivel 0; si no, devuelve falso.
- Si lo encuentra, para cada nivel superior al 0 (bajando desde la altura del nodo) utiliza `CompareAndSet` para marcar la referencia al siguiente en el nivel (reintentando hasta lograrlo)
- Finalmente, hace lo mismo en el nivel 0, quedándose con el valor de retorno de `CaS` para verificar si el borrado fue suyo.
- No hace borrado físico, de eso se encargará `find`.

```
1 boolean remove(T x) {
2     int bottomLevel = 0;
3     Node<T>[] preds =
4         (Node<T>[]) new Node[MAX_LEVEL + 1];
5     Node<T>[] succs =
6         (Node<T>[]) new Node[MAX_LEVEL + 1];
7     Node<T> succ;
8     while (true) {
9         boolean found = find(x, preds, succs);
10        if (!found) { //l. fallo
11            return false;
12        } else {
13            Node<T> nodeToRemove = succs[bottomLevel];
14            for (int level = nodeToRemove.topLevel;
15                level >= bottomLevel+1; level--) {
16                boolean[] marked = {false};
17                succ = nodeToRemove.next[level]
18                    .get(marked);
19                while (!marked[0]) {
20                    nodeToRemove.next[level].compareAndSet(
21                        succ, succ, false, true);
22                    succ = nodeToRemove.next[level]
23                        .get(marked);
24                }
25            }
```

```
26        boolean[] marked = {false};
27        succ = nodeToRemove.next[bottomLevel].get(
28            marked);
29        while (true) {
30            //l. exito
31            boolean iMarkedIt =
32                nodeToRemove.next[bottomLevel]
33                    .compareAndSet(succ, succ, false, true);
34            succ = succs[bottomLevel].next[
35                bottomLevel].get(marked);
36            if (iMarkedIt) {
37                find(x, preds, succs);
38                return true;
39            }
40        }
41    }
42 }
```



- Busca el nodo con el elemento bajando desde el nivel más alto.
- Como en el caso anterior, va manteniendo los arrays de predecesores y sucesores.
- Si encuentra una referencia con marca, significa que el nodo actual debe ser eliminado, por lo que lo enlaza el predecesor con el sucesor

```
1 boolean find(T x, Node<T>[] preds, Node<T>[] succs) {
2     int bottomLevel = 0;
3     int key = x.hashCode();
4     boolean[] marked = {false};
5     boolean snip;
6     Node<T> pred = null, curr = null, succ = null;
7     retry:
8     while (true) {
9         pred = head;
10        for (int level = MAX_LEVEL; level >= bottomLevel;
11            level--) {
12            curr = pred.next[level].getReference();
13            while (true) {
14                succ = curr.next[level].get(marked);
15                while (marked[0]) {
16                    snip = pred.next[level]
17                        .compareAndSet(curr, succ, false, false);
18                    if (!snip) continue retry;
19                    curr = pred.next[level].getReference();
20                    succ = curr.next[level].get(marked);
21                }

```

```
22        if (curr.key < key){
23            pred = curr; curr = succ;
24        } else {
25            break;
26        }
27    }
28    preds[level] = pred;
29    succs[level] = curr;
30 }
31 return (curr.key == key);
32 }
33 }
```

- Hace la búsqueda desde el nivel más alto, bajando cuando se cruza con un elemento mayor o igual al buscado.
- Si encuentra nodos marcados, no mira su clave (pero no los elimina)
- Al llegar al nivel 0, responde según si la clave del elemento actual es la buscada.

```
1 boolean contains(T x) {
2     int bottomLevel = 0;
3     int v = x.hashCode();
4     boolean[] marked = {false};
5     Node<T> pred = head, curr = null, succ = null;
6     for (int level = MAX_LEVEL; level >= bottomLevel; level--) {
7         curr = curr.next[level].getReference();
8         while (true) {
9             succ = curr.next[level].get(marked);
10            while (marked[0]) {
11                curr = pred.next[level].getReference();
12                succ = curr.next[level].get(marked);
13            }
14            if (curr.key < v){
15                pred = curr;
16                curr = succ;
17            } else {
18                break;
19            }
20        }
21    }
22    return (curr.key == v); //linealiza
23 }
```

